

第十三次上机作业

学号：241830137 姓名：朱子健

2026 年 6 月 3 日

1 问题描述

1. 考虑初值问题：

$$y' = t - y^2, \quad y(0) = 1.$$

使用 Runge-Kutta-Fehlberg 方法在区间 $[0, 2]$ 上求解上述问题，其中 参数设定为： $TOL = 10^{-5}$, $h_{\max} = 10^{-1}$, $h_{\min} = 10^{-8}$. 题目要求：

1. 分别用表和图显示数值解 $y(t)$ 在 $t \in [0, 2]$ 上的近似值；
2. 绘制步长 h 随 t 变化图.

2 算法原理

考虑一阶常微分方程初值问题

2.1 引言：龙格-库塔方法

龙格-库塔 (Runge-Kutta, RK) 方法是一系列用于数值近似解决常微分方程 (ODE) 的迭代技术。这些方程通常具有以下形式： $\frac{dy}{dt} = f(t, y)$ 。自适应方法的核心目标是控制局部截断误差，确保在满足指定容忍度 (Tolerance, TOL) 的前提下，实现计算效率的最大化。

2.2 自适应步长和误差估计

自适应方法通常通过比较在下一步时间点计算得到的两个不同阶次的近似解，来估计局部截断误差 (E)。一种常见的技术是比较使用两个不同阶次龙格-库塔公式得到的近似结果（例如，比较四阶和五阶结果）。如果估计出误差 E ，则使用以下公式调整下一步的步长 h_{new} ，以确保满足目标容忍度 TOL ：

$$h_{new} = h_{old} \left(\frac{TOL}{\|E\|} \right)^{1/p} \quad (1)$$

其中， p 是方法当前的阶次（例如，若使用五阶结果来估计四阶误差，则 $p = 5$ ）。

2.3 龙格-库塔-费尔伯格 (RKF) 方法

自适应 RK 方法的一个经典示例就是龙格-库塔-费尔伯格 (Runge-Kutta-Fehlberg, RKF) 方法。该方法巧妙地在这次计算中同时得到两个阶次的估计值：一个为 p 阶的初级解，另一个为 $p+1$ 阶的误差估计。其核心思想是计算误差 $\|E\|$ ，进而调整步长 h

算法 1 ODE 求解的自适应步长控制流程

输入: ODE = $f(t, y)$, 初值 (t_0, y_0) , 结束时间 t_{end} , 容差 ϵ

```
1: 初始化  $t_{current} = t_0, y_{current} = y_0$ 
2: while  $t_{current} < t_{end}$  do
3:   猜测当前步长  $h_{trial}$ 
4:   计算预测值  $\hat{y}_{next}$  和误差估计  $E$  (利用高阶方法估计误差)
5:   if  $E \leq \text{tolerance}$  then
6:     接受当前步长  $h_{accepted} = h_{trial}$ 
7:     更新  $t_{current} \leftarrow t_{current} + h_{accepted}$ 
8:     更新  $y_{current} \leftarrow \hat{y}_{next}$ 
9:     根据误差  $E$  调整下一个步长  $h_{next} = h_{accepted} \cdot \left(\frac{\epsilon}{|E|}\right)^{1/k}$ 
10:  else
11:    拒绝当前步长  $h_{trial}$ 
12:    减小步长  $h_{next} = h_{trial} \cdot \left(\frac{\epsilon}{|E|}\right)^{1/k}$ 
13:  end if
14: end while
15: return  $y_{current}$ 
```

3 结果分析

表 1: RKF 方法求解 $y' = t - y^2$, $y(0) = 1$ 的数值结果

t	$y(t)$
0.0000000	1.0000000
0.1000000	0.91379418
0.2000000	0.85119100
0.3000000	0.80762134
0.4000000	0.77980699
0.5000000	0.76528025
0.6000000	0.76209180
0.7000000	0.76862846
0.8000000	0.78349852
0.9000000	0.80546079
1.0000000	0.83338304
1.1000000	0.86622026
1.2000000	0.90300631
1.3000000	0.94285370
1.4000000	0.98495769
1.5000000	1.02860157
1.6000000	1.07316094
1.7000000	1.11810560
1.8000000	1.16299832
1.9000000	1.20749057
2.0000000	1.25131548

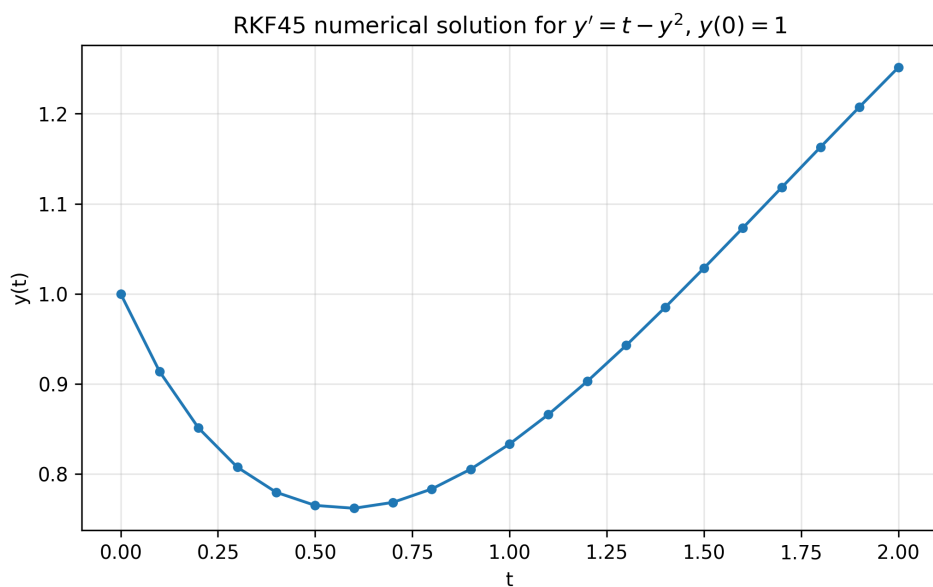


图 1: RKF 方法得到的数值解 $y(t)$

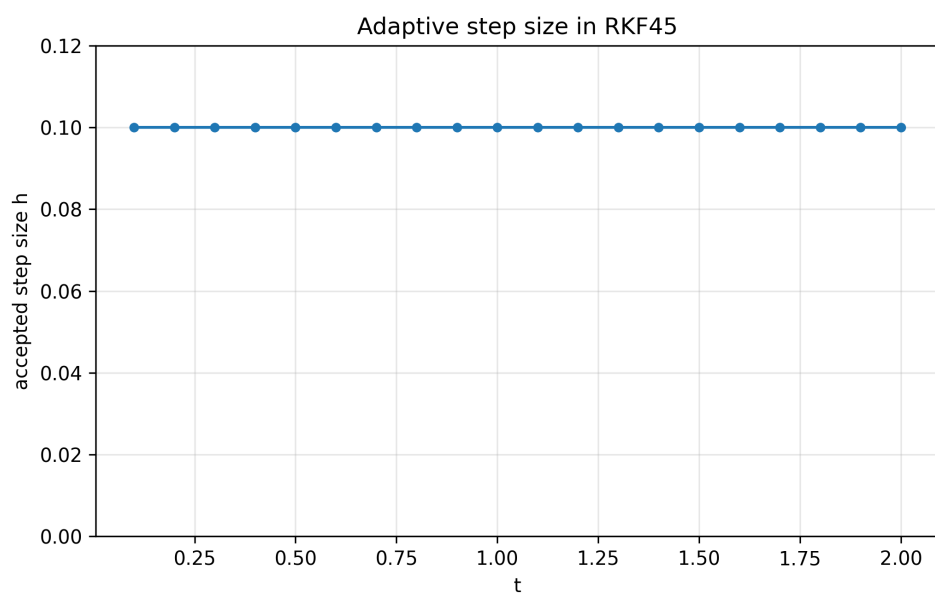


图 2: RKF 方法中步长 h 随 t 的变化

4 附录：Python 程序代码

```
1 """
2 P13: Runge-Kutta-Fehlberg adaptive method for
3  $y' = t - y^2$ ,  $y(0) = 1$ ,  $0 \leq t \leq 2$ .
4
5 This version only solves the ODE and draws the two figures required by the homework
6 :
7 1. numerical solution  $y(t)$  on  $[0, 2]$ ;
8 2. accepted step size  $h$  versus  $t$ .
9
10 It does NOT create or print a table. Put the table in your LaTeX report instead.
11 """
12 from pathlib import Path
13
14 import numpy as np
15 import matplotlib.pyplot as plt
16
17 OUTPUT_DIR = Path(__file__).resolve().parent
18
19
20 def f(t, y):
21     """Right-hand side of the ODE:  $y' = t - y^2$ ."""
22     return t - y**2
23
24
25 def rkf45_step(func, t, y, h):
26     """
27     One Runge-Kutta-Fehlberg 4(5) step.
28
29     The notation follows the textbook/notes:  $K_i = hf(\dots)$ .
30     Returns:
31     y4: fourth-order approximation
32     y5: fifth-order approximation
33     R: local error estimate per unit step,  $|y_5 - y_4|/h$ 
34     """
35     K1 = h * func(t, y)
36     K2 = h * func(t + h / 4, y + K1 / 4)
37     K3 = h * func(t + 3 * h / 8, y + 3 * K1 / 32 + 9 * K2 / 32)
38     K4 = h * func(
39         t + 12 * h / 13,
40         y + 1932 * K1 / 2197 - 7200 * K2 / 2197 + 7296 * K3 / 2197,
41     )
42     K5 = h * func(
43         t + h,
44         y + 439 * K1 / 216 - 8 * K2 + 3680 * K3 / 513 - 845 * K4 / 4104,
45     )
46     K6 = h * func(
```

```

47         t + h / 2,
48         y - 8 * K1 / 27 + 2 * K2 - 3544 * K3 / 2565
49         + 1859 * K4 / 4104 - 11 * K5 / 40,
50     )
51
52     # Fourth-order formula
53     y4 = y + 25 * K1 / 216 + 1408 * K3 / 2565 + 2197 * K4 / 4104 - K5 / 5
54
55     # Fifth-order formula
56     y5 = y + 16 * K1 / 135 + 6656 * K3 / 12825 + 28561 * K4 / 56430 - 9 * K5 / 50 +
57         2 * K6 / 55
58
59     R = abs(y5 - y4) / abs(h)
60     return y4, y5, R
61
62 def solve_rkf45(func, a, b, y0, tol=1e-5, hmax=1e-1, hmin=1e-8, safety=0.84):
63     """
64     Adaptive_RKF45_solver.
65
66     Step_acceptance_rule:
67     accept the step if R<=tol;
68     otherwise shrink h and retry.
69
70     Step-size_update:
71     delta=0.84*(tol/R)^(1/4),
72     with the usual clipping used in the notes/algorithm.
73     """
74     t = float(a)
75     y = float(y0)
76     h = float(hmax)
77
78     t_values = [t]
79     y_values = [y]
80     h_values = []      # h used by each accepted step
81     t_for_h = []      # endpoint t after each accepted step
82     r_values = []
83
84     max_trials = 1_000_000
85     trials = 0
86
87     while t < b and trials < max_trials:
88         trials += 1
89
90         # Shorten the last step so that the final point is exactly b.
91         h_try = min(h, b - t)
92
93         if h_try < hmin:
94             raise RuntimeError(f"Minimum_h_exceeded_near_t={t:.16g}.")
95

```

```

96     y4, y5, R = rkf45_step(func, t, y, h_try)
97
98     if R <= tol:
99         # The textbook algorithm accepts the fourth-order value.
100         t = t + h_try
101         y = y4
102
103         t_values.append(t)
104         y_values.append(y)
105         h_values.append(h_try)
106         t_for_h.append(t)
107         r_values.append(R)
108
109     # Update the trial step size.
110     if R == 0:
111         delta = 4.0
112     else:
113         delta = safety * (tol / R) ** 0.25
114
115     if delta <= 0.1:
116         h = 0.1 * h_try
117     elif delta >= 4.0:
118         h = 4.0 * h_try
119     else:
120         h = delta * h_try
121
122     h = min(h, hmax)
123
124     if h < hmin:
125         raise RuntimeError(f"Minimum h exceeded after updating h near t={t:.16g}
126                             {t:.16g}")
127
128     if trials >= max_trials:
129         raise RuntimeError("Too many trials; the adaptive loop did not finish.")
130
131     return np.array(t_values), np.array(y_values), np.array(t_for_h), np.array(
132         h_values), np.array(r_values)
133
134 def draw_solution(t, y, filename_pdf="P13_solution_curve.pdf", filename_png="
135     P13_solution_curve.png"):
136     """Draw numerical solution y(t)."""
137     plt.figure(figsize=(7, 4.5))
138     plt.plot(t, y, marker="o", markersize=4, linewidth=1.5)
139     plt.xlabel("t")
140     plt.ylabel("y(t)")
141     plt.title(r"RK45 numerical solution for $y'=t-y^2$, $y(0)=1$")
142     plt.grid(True, alpha=0.3)
143     plt.tight_layout()
144     plt.savefig(OUTPUT_DIR / filename_pdf)

```

```

143     plt.savefig(OUTPUT_DIR / filename_png, dpi=300)
144     plt.show()
145
146
147 def draw_stepsize(t_for_h, h_values, filename_pdf="P13_stepsize_curve.pdf",
148     filename_png="P13_stepsize_curve.png"):
149     """Draw accepted step size h versus t."""
150     plt.figure(figsize=(7, 4.5))
151     plt.plot(t_for_h, h_values, marker="o", markersize=4, linewidth=1.5)
152     plt.xlabel("t")
153     plt.ylabel("accepted_step_size_h")
154     plt.title("Adaptive_step_size_in_RKF45")
155     plt.ylim(0, 1.2 * np.max(h_values))
156     plt.grid(True, alpha=0.3)
157     plt.tight_layout()
158     plt.savefig(OUTPUT_DIR / filename_pdf)
159     plt.savefig(OUTPUT_DIR / filename_png, dpi=300)
160     plt.show()
161
162 if __name__ == "__main__":
163     TOL = 1e-5
164     HMAX = 1e-1
165     HMIN = 1e-8
166
167     t, y, t_for_h, h_values, r_values = solve_rkf45(
168         f, a=0.0, b=2.0, y0=1.0, tol=TOL, hmax=HMAX, hmin=HMIN
169     )
170
171     draw_solution(t, y)
172     draw_stepsize(t_for_h, h_values)
173
174     print("Figures saved:")
175     print("P13_solution_curve.pdf / P13_solution_curve.png")
176     print("P13_stepsize_curve.pdf / P13_stepsize_curve.png")

```